AI 时代计算机核心基础课程知识体系梳理与教学素材建设计划

——项目实施方案（讨论稿）

一、项目缘起

近两年，大语言模型、智能体、AI Coding 等技术发展很快，也带来了一个越来越常见的问题：在 AI 工具已经能够生成代码、分析程序甚至完成较复杂开发任务的情况下，计算机专业为什么还要花大量时间学习数据结构、计算机组成原理、操作系统和计算机网络这些基础课程?

这个问题表面上是在讨论课程是否“过时”，实质上反映出当前基础课程教学中的一个长期问题：很多学生能够分别学习链表、Cache、虚拟内存、进程、TCP 等知识，但并没有真正建立起这些知识之间的整体联系，也不容易看到它们在现代计算机系统中的实际作用。

事实上，无论是大语言模型的训练与推理，还是 RAG、Agent、Coding Agent、Harness 等新的 AI 系统，其底层仍然离不开数据组织、存储层次、进程与并发、内存管理、网络通信、分布式计算等经典计算机知识。同时，机器学习和人工智能技术也正在反过来影响算法设计、体系结构、操作系统优化、网络调度等传统领域。

因此，本项目希望以当前 AI 技术发展为切入点，重新梳理计算机核心基础知识，重点回答三个问题：

1. 数据结构、计算机组成原理、操作系统和计算机网络四门课程之间，本身有哪些重要的知识联系和支撑关系；
2. 这些基础知识如何支撑当前的大模型、Agent、Harness 及其他 AI 系统，其中哪些知识在真实系统中仍然发挥着关键作用；
3. 人工智能和机器学习的发展，又在怎样影响算法、计算机体系结构、操作系统和网络等传统计算机领域。

项目不以简单增加“AI 案例”为目标，而是希望逐步建立一套能够解释传统知识、连接现代技术、支持课堂教学和学生实践的素材体系。在此基础上，再考虑形成专题网站、教学资源库、课程内容乃至教材。

二、项目目标

项目第一阶段不追求“大而全”，重点是把整体框架搭起来，并形成一批经得住推敲的内容。

计划逐步完成以下几项工作。

第一，梳理四门核心基础课程中的关键知识点，重点识别不同课程之间具有明显联系的内容，形成一张比较清晰的基础知识关系图。

第二，从当前大模型和 AI 系统出发，寻找基础知识在真实技术中的应用。重点关注 LLM 训练与推理、RAG、Agent、Coding Agent、Harness、AI 基础设施等方向，形成一批能够回答“这个知识今天有什么用”的典型案例。

第三，整理 AI 技术在算法、体系结构、操作系统、网络等方向中的新应用，形成“基础知识支撑 AI”和“AI 反过来推动计算机系统发展”两条相互衔接的内容线。

第四，在梳理过程中同步建设统一的素材库。每一个重要知识点或案例都尽量保留来源、相关论文或系统、代码、图示和可进一步开发的实验，避免最后只留下文字总结。

第五，在素材逐渐成熟后，选择其中适合教学的内容开发成实验、专题讲义或课堂材料，并根据实际使用情况调整整个知识体系。项目成熟后，可进一步考虑整理形成一本面向本科生和年轻开发者的书。

三、总体内容框架

整个项目暂时分为三个相互关联的部分。

（一）四门基础课程之间的知识联系

首先不急于加入 AI，而是把四门课本身的逻辑关系理顺。

数据结构主要解决数据如何组织以及如何高效计算的问题；计算机组成原理关注程序如何在处理器、存储器和设备上执行；操作系统研究如何管理 CPU、内存、文件和设备，以及如何支持多个程序协同运行；计算机网络解决不同计算机和程序之间如何通信的问题。

实际系统中的很多问题往往同时涉及几门课程。例如：

- 队列既是数据结构，也是操作系统调度和网络缓冲的重要基础；
- LRU 既可以作为数据结构问题理解，也直接关联缓存和页面置换；
- 多级页表涉及树形结构、地址转换、MMU 和虚拟内存；
- 路由问题本身就是典型的图问题；
- 锁和同步既需要理解操作系统中的并发，也需要理解处理器提供的原子操作；
- 高性能网络通信最终还会涉及 DMA、PCIe、内存访问和设备管理。

这一部分的重点不是重新总结教材，而是寻找这些“连接点”。

（二）基础知识对现代 AI 系统的支撑

这是项目的重点。

以当前学生最关注的大语言模型、RAG、Agent 和 AI 开发工具为线索，反向分析其中涉及的基础计算机知识。

例如，大模型推理中的 KV Cache 管理可以与操作系统中的分页和虚拟内存联系起来；FlashAttention 可以与 Cache、存储层次和数据搬运联系起来；大模型分布式训练涉及带宽、延迟、集合通信和 RDMA；Agent 中的工具执行又会涉及进程、文件系统、权限、网络、RPC 和安全隔离。

这类案例要尽量落实到真实论文、真实开源系统和真实工程实现，而不是停留在“某个知识可能对 AI 有用”的层面。

这一部分后续可以重点围绕以下几个方向展开：

- LLM 训练与推理；
- RAG 和向量检索；

- Agent 与多 Agent 系统；

- Coding Agent；

- Agent Harness 及运行环境；

- AI 基础设施和分布式训练；

- 模型服务与高性能推理。

随着技术发展，具体方向可以调整，不需要在项目初期全部确定。

（三）AI 对传统计算机领域的反向影响

除了研究传统知识如何支撑 AI，也需要关注 AI 如何反过来影响计算机科学和系统研究。

目前可以关注的方向包括：

- AI 辅助算法发现与算法优化；

- Learned Index 等新的数据组织和索引方法；

- AI 辅助芯片设计和体系结构优化；

- AI 辅助编译与性能优化；

- AI 参与操作系统资源调度和参数优化；

- 机器学习用于网络路由、拥塞控制和流量预测。

这一部分需要特别注意区分已经进入实际系统的方法和仍然处于研究阶段的方法，不把尚未成熟的研究包装成既成事实。

四、素材库建设方式

项目开始后，所有成员尽量不要各自保存零散文档，而是按照统一方式沉淀材料。

第一阶段建议以 Markdown、表格和简单的 JSON/YAML 文件为主，不急于建设复杂平台。等素材数量和结构稳定以后，再考虑做知识图谱、检索系统或者 RAG 应用。

每一个比较重要的素材至少需要记录以下内容：

- 所属课程；

- 基础知识点；

- 与其他基础课程的联系；

- 对应的 AI 技术或系统；

- 两者之间的具体联系；

- 真实论文、开源项目或官方技术资料；

- 是否适合课堂讲解；

- 是否可以设计实验；

- 当前成熟程度；

- 仍然存在的问题或需要进一步核实的内容。

其中最重要的是“为什么二者有关”。如果这一点说不清楚，就暂时不要作为正式案例收入核心素材库。

对于重点案例，后续可以进一步整理成完整的案例卡，包括背景、传统知识、现代系统问题、技术实现、相关代码和实验设计。

五、素材选择原则

为了避免项目后期出现大量内容但真正可用的材料不多，素材收集阶段需要有基本标准。

优先收录以下几类内容：

1. 能够明确连接一个或多个基础知识点；

2. 来自真实系统、经典论文或有影响力的开源项目；

3. 能够解释一个具体技术问题，而不是泛泛讨论 AI 发展；

4. 本科生经过适当补充知识后能够理解；

5. 有机会进一步设计成小型实验或演示。

对一些很有意思但目前证据不足、技术尚未成熟的内容，可以先放入“观察素材”，不急于进入正式案例库。

整个项目宁可先形成二三十个扎实的案例，也不追求快速生成几百条表面相关的材料。

六、工作组织

项目建议采用“课程负责人 + AI 方向负责人 + 综合整理”的协作方式。

可以分别安排成员重点跟踪数据结构、计算机组成原理、操作系统和计算机网络，同时安排成员持续关注大模型、Agent、Harness 以及 AI 系统领域的发展。

但在实际整理案例时，尽量避免一个人从头做到尾。重要案例至少由基础课程方向和 AI 方向各有一名成员参与，必要时再邀请其他方向成员讨论。

例如，PagedAttention 一类案例可以由操作系统方向和 LLM 系统方向共同整理；FlashAttention 适合由计算机组成原理方向与 AI 方向共同完成；MCP 或 Agent Harness 则可能同时涉及操作系统和网络。

这样既可以提高内容准确性，也能够自然形成跨课程联系。

七、近期工作安排

项目启动后的第一阶段建议控制在三个月左右。

第一阶段：搭框架

先用两周左右时间完成四门课程核心知识点梳理。

这一轮不需要把教材所有章节都列进去，每门课选择大约 15—20 个真正重要、能够与其他课程或现代系统产生联系的知识点即可。

在此基础上形成第一版知识点清单和课程间关系图。

第二阶段：寻找关键连接

接下来集中寻找“传统知识—现代 AI 系统”的连接。

初期目标不是数量，而是找到一批有代表性的案例。建议首先完成 20—30 个较强的候选案例，并判断其中哪些值得深入。

这一阶段同时建立参考资料库，包括论文、官方技术文档、开源项目和相关代码。

第三阶段：做深重点案例

从候选案例中选择大约 15—20 个重点内容进行深入整理。

每个重点案例至少要把以下问题讲清楚：

- 传统课程里学的是什么；
- 这个知识最初解决什么问题；
- 现代 AI 系统遇到了什么类似或相关的问题；
- 实际系统是如何解决的；
- 传统知识和现代技术之间到底是什么关系；
- 是否能够设计一个本科生可以完成的小实验。

完成这一阶段后，项目应该已经能够形成第一批可以真正用于课堂或分享的材料。

第四阶段：补充 AI 反哺内容和实验

在前面的主体框架比较稳定后，再系统整理 AI 用于算法、体系结构、操作系统和网络的案例。

同时从已有案例中挑选适合实现的内容，逐步建设一个小型实验库。

首批实验不宜复杂，尽量控制在本科生能够在 1—3 小时内完成或理解的程度。实验的主要目的不是做大型工程，而是帮助学生真正看到传统知识和现代 AI 系统之间的联系。

八、阶段性成果

项目第一轮完成后，希望至少形成以下几项成果：

1. 一份四门基础课程核心知识点清单；
2. 一张能够体现课程之间联系的知识关系图；
3. 一张“基础知识—AI 技术—真实系统”的对应关系表；
4. 20—30 个经过筛选的现代 AI 系统案例；

5. 10 个左右较完整的重点案例材料；

6. 若干可运行的小型实验；

7. 一套结构比较稳定的素材库。

这些成果不要求一次性达到最终状态。第一版的主要作用，是让整个团队拥有一个共同的知识框架和工作方式。

九、后续发展

项目后期如何展开，可以根据前期素材质量和实际教学效果再决定。

如果知识体系逐渐稳定，可以建设可检索的知识库和专题网站；如果实验效果比较好，可以形成针对基础课程的补充实验；如果内容已经形成比较完整的逻辑，则可以逐步整理为教材或面向普通读者的技术读物。

可能的成书方向可以暂定为：

《AI 时代为什么还要学计算机基础——从数据结构、计算机系统到 LLM、Agent 与 Harness》

但现阶段不以“写书”为直接任务。优先把知识关系、真实案例和实验做好。书应该是这些内容逐渐成熟之后自然形成的结果，而不是反过来按照目录去填内容。

十、项目实施中的几个基本原则

第一，**以真实问题为主，不为融合而融合**。

如果某个基础知识点与 AI 的联系非常牵强，就没有必要一定找到一个 AI 案例。

第二，**以证据为基础**。

重要结论尽量能够回到论文、官方技术资料、开源系统或实际代码。

第三，**先少后多**。

前期首先做出一批真正好的案例，再逐渐扩充。

第四，**边做边调整**。

当前提出的知识范围、AI 方向和案例数量都是第一版方案。随着项目推进以及 AI 技术的发展，可以调整重点，不需要机械执行最初计划。

第五，**素材建设优先于平台建设**。

在内容没有形成之前，不投入太多时间开发网站、知识图谱或复杂系统。

第六，**最终要能够用于教学**。

无论一个技术多么前沿，如果无法帮助学生理解基础知识，也不一定适合作为本项目的核心素材。

十一、第一阶段结束时希望达到的状态

三个月以后，不要求项目已经形成完整教材，也不要求所有技术方向都覆盖。

比较理想的状态是：团队已经能够清楚回答“四门基础课为什么是一个整体”，已经找到一批能够说明“传统计算机基础如何支撑今天 AI 系统”的真实案例，并且其中若干案例已经能够用于课堂讲解或者学生实验。

在这个基础上，再决定下一阶段重点扩展哪些内容。

整个项目可以始终围绕一个简单的问题展开：

今天一个学生正在学习的数据结构、Cache、虚拟内存、进程、调度、Socket 和 TCP，在他未来真正开发和理解 LLM、Agent 以及下一代 AI 系统时，究竟会在哪里再次遇到？

如果这件事情能够讲清楚，这个项目的核心目标就已经实现了一半。